# Research Proposal: Detection and Segmentation of Cut Edges

Pablo Ruíz Morán
pruizm01@estudiantes.unileon.es

David Morán Gorgojo
dmorag03@estudiantes.unileon.es

Miguel Sánchez Rodriguez
msancr11@estudiantes.unileon.es

Jaime Alvarado Fernandez
jalvaf08@estudiantes.unileon.es

March 24, 2026

**Abstract**

This research project addresses the automated detection and metric estimation of cutting edges in rubber strip assembly within tyre manufacturing, developed in the context of the **Michelin Challenge 2** proposed by *Michelin Aranda de Duero*. The core challenge consists of accurately localizing cut borders and estimating their spatial positions in millimetres from smartphone images captured under variable perspectives, illumination, and limited annotated data, where precision and robustness are critical for ensuring product quality.

To tackle this problem, we review recent methods (2024–2026) spanning object detection, image segmentation, anomaly detection, and classical edge detection, and propose four complementary computer vision pipelines. The first is **Grounded-SAM**, which combines Grounding DINO for open-vocabulary detection with SAM for precise mask generation, enabling zero-shot segmentation without task-specific training. The second is a **YOLOv9 + EfficientSAM** pipeline, which integrates real-time object detection with lightweight promptable segmentation for fine-tunable performance on the target dataset. The third is an **anisotropic multi-scale edge detector (MAMDD)**, a training-free classical approach that captures thin separation lines under noise and low contrast. The fourth is an **adaptive classical pipeline** based on improved Canny edge detection with Otsu thresholding, QR-based homography calibration, and Probabilistic Hough Transform, deployable entirely on CPU without any training data.

All four pipelines incorporate QR-code-based spatial calibration to correct perspective distortion and provide millimetre-level distance measurements. Together, they cover a spectrum from zero-shot deep learning to fully classical approaches, offering flexible trade-offs between accuracy, computational cost, and data requirements for real industrial deployment.

***Keywords:*** *Cut Edge Detection, Object Detection, Image Segmentation, Grounded-SAM, YOLOv9, Classical Edge Detection, Industrial Inspection, Michelin Challenge*

## 1 Introduction

Automatic detection and precise localization of cutting edges in industrial images is a key problem in modern manufacturing, driven by the increasing demand for automated quality control systems. In tyre assembly processes, rubber strips are placed over a cylindrical drum, cut, and subsequently joined at their edges. Currently, these tasks are performed manually by workers on the production floor, introducing variability, human error, and limiting the scalability of the process. Automating this inspection step would directly reduce operational costs and improve product consistency across production lines.

Precise cut edge localization is particularly challenging in this setting. Images are captured using smartphones by different workers from variable positions and angles, requiring robustness to perspective distortion, illumination changes, and varying camera distance. The rubber strip and its cut edges often present low contrast with respect to the background, and an additional requirement is to provide metric measurements in millimetres rather than pixel coordinates, which necessitates a reliable pixel-to-metric calibration procedure based on QR codes placed on the inspection surface.

These challenges sit at the intersection of several active research directions in Deep Learning and Computer Vision. Recent advances have produced powerful methods for object detection and segmentation: convolutional approaches such as YOLOv8 [1] offer real-time detection, while transformer-based detectors such as RT-DETRv4 [2] achieve state-of-the-art results on standard benchmarks. Vision-language models such as Grounding DINO [3] extend detection to open-vocabulary settings without task-specific retraining, and the Segment Anything Model (SAM) [4] enables promptable pixel-level segmentation for arbitrary objects. However, most of these methods are evaluated on general-purpose benchmarks, and their applicability to industrial inspection with limited labeled data, domain shift, and strict metric accuracy requirements remains an open challenge.

The specific objectives of this work are: (i) to accurately detect and segment the cut edges of rubber strips from unconstrained smartphone images; (ii) to estimate their spatial positions and mutual distances in millimetres using QR-based geometric calibration; and (iii) to evaluate the suitability of both deep learning and classical approaches under the data and deployment constraints of a real industrial environment. To this end, we review twenty recent methods (2024–2025) and propose four complementary pipelines: Grounded-SAM for zero-shot open-vocabulary segmentation, YOLOv9 + EfficientSAM for fine-tunable detection and segmentation, an anisotropic multi-scale edge detector (MAMDD) for training-free classical edge localization, and an adaptive Canny-based pipeline with QR-guided homography calibration for fully CPU-deployable metric estimation.

The rest of the document is organized as follows. Section 2 presents the literature review. Section 3 describes the four proposed pipelines. Section 4 presents the datasets, Section 5 defines the evaluation metrics, Section 6 covers the available Python implementations, and Section 7 lists individual contributions.

## 2 Literature Review

The following section reviews recent methods (2024–2025) relevant to the **Michelin Challenge 2**, organized into four groups: deep learning-based object detection and segmentation, anomaly detection, classical computer vision, and camera calibration. The goal of this review is to identify the most suitable approaches for cut edge detection and metric estimation under the constraints of the challenge: limited annotated data, variable acquisition conditions, and the need for millimetre-level accuracy without specialized hardware.

Among the reviewed deep learning methods, the strongest candidates for this problem

are **Grounded-SAM** [3], which achieves $48.7$ mAP in zero-shot settings on 25 diverse real-world datasets without task-specific training; **YOLOv9 + EfficientSAM**, combining $55.6\%$ $AP_{50:95}$ detection with lightweight promptable segmentation at 47 img/s; and **Anomaly-DINO** [5], which reaches $96.6\%$ AUROC in one-shot anomaly detection without any fine-tuning. On the classical side, the **MAMDD** edge detector [6] and the **improved Canny pipeline** [7] stand out for their robustness to noise and direct applicability without training data.

### 2.1 Deep Learning: Object Detection and Segmentation

Jocher et al. [1] present YOLOv8, a next-generation object detection model developed by Ultralytics. The architecture improves previous YOLO versions by integrating a CSPNet backbone for enhanced feature extraction, an FPN+PAN neck for multi-scale feature aggregation, and an anchor-free detection head that simplifies bounding box prediction and reduces computational complexity. The model also introduces improved training strategies such as advanced data augmentation techniques (mosaic and mixup), focal loss for classification, and mixed-precision training. Benchmark evaluations on Microsoft COCO and Roboflow 100 demonstrate that YOLOv8 achieves $71.5\%$ mAP@0.5 with the YOLOv8x variant at 11.5 ms inference time.

Liao et al. [2] propose RT-DETRv4, a real-time object detection framework that improves DETR-based detectors by distilling semantic knowledge from Vision Foundation Models into lightweight architectures. The method introduces a Deep Semantic Injector (DSI) module and a Gradient-guided Adaptive Modulation (GAM) strategy that dynamically balances semantic transfer during training. Evaluated on COCO 2017, RT-DETRv4 achieves AP scores of 49.7, 53.5, 55.4, and 57.0 for the S, M, L, and X variants at speeds of 273, 169, 124, and 78 FPS respectively.

Kienzle et al. [8] propose Segformer++, an efficient transformer-based architecture for high-resolution semantic segmentation that adapts token-merging strategies to the Segformer framework without requiring model retraining. Two configurations are introduced: Segformer++HQ, oriented towards quality preservation, and Segformer++fast, oriented towards speed. Evaluated on Cityscapes and ADE20K, Segformer++HQ achieves a mIoU of $82.31$ on Cityscapes with a $1.61\times$ inference speedup, while Segformer++fast reaches a $1.94\times$ speedup with only a marginal performance drop.

Cavagnero et al. [9] propose PEM (Prototype-based Efficient MaskFormer), a transformer-based architecture for image segmentation that introduces a prototype-based cross-attention mechanism to reduce computational burden, together with an efficient multi-scale feature pyramid network combining deformable convolutions and context-based self-modulation. Evaluated on Cityscapes and ADE20K, PEM achieves $79.9$ mIoU at 13.8 FPS for semantic segmentation and $61.1$ PQ at 13.8 FPS for panoptic segmentation.

Hou et al. [10] propose Relation-DETR, a transformer-based object detector that introduces an explicit position relation prior to improve convergence speed and detection accuracy in DETR architectures. The method incorporates a position relation encoder that models pairwise interactions between bounding boxes and uses this information as an attention bias for progressive attention refinement. Evaluated on COCO 2017, Relation-DETR achieves $51.7$ AP with ResNet-50, outperforming DINO by $+2.0$ AP and reaching over 40 AP after only 2 training epochs.

Heidari et al. [11] introduce ViLU-Net, a hybrid encoder–decoder architecture that integrates Vision-LSTM (xLSTM) blocks within a U-Net structure to model long-range spatial dependencies. The model combines CNN layers for local feature extraction with xLSTM modules for global contextual information. Evaluated on a CT dataset of retroperitoneal tumours and the FLARE dataset, ViLU-Net achieves a DSC of $0.9309$ and IoU of $0.8720$, outperforming methods such as nnU-Net, SwinUNETR, and U-Mamba.

Ren et al. [3] introduce Grounded-SAM, a modular framework that combines Grounding DINO, an open-set detector capable of identifying objects from free-form text prompts, with the Segment Anything Model (SAM), which generates precise pixel-level masks. Grounding DINO detects candidate regions and outputs bounding boxes, which are then used as prompts by SAM to produce segmentation masks. Evaluated on the Segmentation in the Wild (SegInW) benchmark — comprising 25 diverse real-world datasets — the method achieves $48.7$ mAP in zero-shot settings, demonstrating strong generalization for open-world vision tasks without additional training.

Cheng et al. [12] propose YOLO-World, a real-time open-vocabulary object detector that extends the YOLO architecture by incorporating linguistic information through a CLIP-based text encoder and a Re-parameterizable Vision-Language Path Aggregation Network (RepVL-PAN). The model achieves $35.4$ AP on the LVIS dataset in zero-shot settings at 52 FPS, demonstrating strong performance in open-world scenarios with objects not explicitly included in the training set.

Wyatt et al. [13] propose an unsupervised anomaly detection approach for defect localization in ultrasonic non-destructive testing using a generative diffusion model. Their method relies on AnoDDPM trained exclusively on defect-free data to reconstruct normal ultrasonic wave propagation patterns. During inference, defects are detected by computing the difference between the input image and the reconstructed output, producing anomaly maps that highlight defective regions. Applied to a Laser Ultrasonic Visualization Testing (LUVT) dataset for aluminium inspection, the method achieves an AUROC of 0.909, outperforming VAE ($0.800$) and f-AnoGAN ($0.755$), and reaching Precision $0.817$ and Recall $0.820$.

Ren et al. [14] introduce Grounding DINO 1.5, an advanced open-vocabulary detection model that extends the original Grounding DINO framework by combining visual features with textual prompts through a transformer-based architecture. Trained on large-scale grounding datasets, the Pro model achieves $54.3$ AP on COCO and $55.7$ AP on LVIS-minival in zero-shot settings, while an optimised Edge variant reaches 75.2 FPS on A100 with TensorRT.

A summary of the reviewed methods is provided in Table 1.

| Method | Year | Description | Dataset | Main Metric | Code | Speed |
|---|---|---|---|---|---|---|
| YOLOv8 [1] | 2024 | Anchor-free CNN detector with CSPNet backbone and FPN+PAN neck | COCO, Roboflow 100 | 71.5% mAP@0.5 | Yes | 11.5 ms |
| RT-DETRv4 [2] | 2025 | DETR detector with VFM distillation via DSI and GAM | COCO 2017 | 49.7–57.0 AP | Yes | 78–273 FPS |
| Segformer++ [8] | 2024 | Efficient transformer segmentation via token merging | Cityscapes, ADE20K | 82.31 mIoU | Yes | $1.61\times$ speedup |
| PEM [9] | 2024 | Prototype-based efficient MaskFormer | Cityscapes, ADE20K | 61.1 PQ / 79.9 mIoU | Yes | 13.8–35.7 FPS |
| Relation-DETR [10] | 2024 | DETR with explicit position relation prior | COCO 2017 | 51.7 AP | Yes | N/R |
| ViLU-Net [11] | 2025 | Hybrid U-Net with CNN and xLSTM blocks | CT Tumour, FLARE | DSC 0.9309 / IoU 0.8720 | Yes | N/R |
| Grounded-SAM [3] | 2024 | Open-vocabulary detection + segmentation pipeline | SegInW (zero-shot) | 48.7 mAP | Yes | N/R |
| YOLO-World [12] | 2024 | Real-time open-vocabulary YOLO with RepVL-PAN | LVIS | 35.4 AP | Yes | 52.0 FPS |
| Grounding DINO 1.5 [14] | 2024 | Advanced open-vocabulary detector with ViT-L backbone | COCO, LVIS | 54.3 AP / 55.7 AP | Yes | 5.5–75.2 FPS |
| AnoDDPM [13] | 2024 | Unsupervised anomaly detection with diffusion models | LUVT | AUROC 0.909 | Yes | N/R |

Table 1: Summary of the reviewed deep learning methods for object detection and segmentation. N/R = Not Reported.

Ravi et al. [15] present SAM 2 (Segment Anything Model 2), a foundation model developed by Meta FAIR for promptable visual segmentation in images and videos. The architecture consists of a hierarchical Hiera image encoder pretrained with MAE, a prompt encoder accepting points, bounding boxes or masks, a two-way transformer mask decoder, and a streaming memory module for segmentation propagation across frames. Trained on the SA-V dataset comprising 50.9K videos and 35.5M masks, SAM 2 achieves a J&F of $76.6\%$ on MOSE val, while being $6\times$ faster than its predecessor on image segmentation benchmarks, obtaining $61.9\%$ mIoU across 23 zero-shot datasets.

Xiong et al. [16] propose EfficientSAM, a lightweight version of SAM that addresses

the high computational cost of the original model. Their approach, SAMI (SAM-leveraged Masked Image Pretraining), trains lightweight ViT encoders to reconstruct feature embeddings from SAM's ViT-H encoder, transferring representational knowledge into compact models. EfficientSAM-S reduces SAM's parameter count by $\sim 20\times$ (25M vs 636M) while achieving $44.4$ AP on zero-shot instance segmentation on COCO at 47 images per second, compared to SAM's 2 images per second.

Wang et al. [17] propose YOLOv9, a real-time object detection model that addresses the information bottleneck problem through two contributions: Programmable Gradient Information (PGI), which preserves complete input information during training without adding inference cost, and the Generalized Efficient Layer Aggregation Network (GELAN), a lightweight architecture based on CSPNet and ELAN. Evaluated on MS COCO, YOLOv9-E achieves $55.6\%$ $\text{AP}_{50:95}$ with 57.3M parameters, reducing parameter count by $49\%$ and computation by $43\%$ compared to YOLOv8-X while improving AP by $0.6\%$.

Shi et al. [18] propose YOLOv8-HDSA, an improved YOLOv8-based instance segmentation algorithm for industrial carbon block recognition. The architecture introduces a Selective Reinforcement Feature Fusion Module (SRFF) using Hadamard product and dilated convolution to suppress background noise, a convolutional self-attention mechanism for fine-grained edge detail extraction, and Focaler-IoU as a loss function to dynamically adjust sample weights. Evaluated on a real industrial dataset, YOLOv8-HDSA improved recognition accuracy by $7.2\%$ and segmentation accuracy by $3.8\%$ over the baseline YOLOv8.

Damm et al. [5] propose AnomalyDINO, a training-free framework for one-shot and few-shot industrial anomaly detection. The method uses DINOv2 as a patch-level feature extractor and computes distances between patch embeddings of test images and normal reference images to produce anomaly scores and pixel-level segmentation maps, without requiring fine-tuning or additional training data. Evaluated on MVTec-AD, AnomalyDINO achieves an AUROC of $96.6\%$ in the one-shot setting, improving over the previous state of the art by $3.5\%$.

Jiang et al. [19] propose a zero-shot industrial anomaly detection framework that fuses CLIP's global semantic embeddings with DINOv2's multi-scale structural features through a Dual-Modality Attention (DMA) mechanism. A complementary Stabilized Attention-based Pooling (SAP) module suppresses normal background dilution using self-generated anomaly heatmaps. Evaluated across seven industrial benchmarks, the framework achieves an average image-level AUROC of $93.4\%$, AP of $94.3\%$, pixel-level AUROC of $96.9\%$, and AUPRO of $92.4\%$, outperforming WinCLIP, AnomalyCLIP, and Crane.

Xiong et al. [20] propose SAM2-UNet, a U-shaped segmentation framework that lever-

ages the hierarchical Hiera backbone of SAM2 as encoder. Lightweight adapters with a bottleneck MLP design are inserted before each Hiera block for parameter-efficient fine-tuning while keeping the encoder frozen. The decoder follows the classic U-Net design with deep supervision trained with weighted IoU and binary cross-entropy losses. Evaluated across 18 datasets on five benchmarks, SAM2-UNet achieves a mDice of $92.8\%$ on Kvasir-SEG and an S-measure improvement of $4.8\%$ over FEDER on CAMO.

Shi et al. [21] propose a semi-supervised segmentation framework for industrial surface defect detection based on a Two-Branch Perturbation Cross Pseudo Supervision model (TPcps). One branch applies image-level perturbations while the second perturbs the feature space via random channel dropout, with both branches generating mutual pseudo-labels. The network combines a lightweight MobileNetv2 backbone with DeepLabv3Plus and a dynamic threshold strategy. Evaluated on KolektorSDD and five self-built datasets, the method achieves a mIoU $4.39\%$ higher than the state of the art at 64.59 FPS.

Liao et al. [22] present a comprehensive survey of learning-based camera calibration techniques, categorizing existing methods into four models: pinhole, distortion, cross-view, and cross-sensor. The survey reviews deep learning strategies for intrinsic and extrinsic parameter estimation, lens distortion correction, and homography estimation, and compiles a holistic calibration dataset covering synthetic and real-world images. This work is directly relevant to the Michelin Challenge, where QR-based calibration from smartphone images requires robust geometric estimation under perspective distortion.

Heckler-Kram et al. [23] introduce MVTec AD 2, a new benchmark for unsupervised anomaly detection addressing the saturation of existing datasets. The benchmark comprises eight scenarios with over 8000 high-resolution images covering challenging cases including transparent objects, dark-field illumination, high variance in normal data, and extremely small defects. Comprehensive evaluation shows that state-of-the-art methods remain below $60\%$ average AU-PRO, demonstrating the benchmark's discriminatory power.

A summary of the reviewed methods is provided in Table 2.

| Method | Year | Description | Dataset | Main Metric | Code | Speed |
|---|---|---|---|---|---|---|
| SAM 2 [15] | 2024 | Transformer with streaming memory for promptable segmentation | MOSE val, SA-23 | J&F 76.6% / mIoU 61.9% | Yes | 43.8 FPS |
| EfficientSAM [16] | 2023 | Lightweight SAM via SAMI pretraining with ViT-Tiny/Small encoders | COCO, LVIS | 44.4 AP (zero-shot) | Yes | 47–54 img/s |
| YOLOv9 [17] | 2024 | Real-time detector with PGI and GELAN | MS COCO | 55.6% $AP_{50:95}$ | Yes | N/R |
| YOLOv8-HDSA [18] | 2025 | Improved YOLOv8 with SRFF and Focaler-IoU for industrial segmentation | Industrial carbon block | +7.2% accuracy over YOLOv8 | Yes | N/R |
| AnomalyDINO [5] | 2024 | Training-free few-shot anomaly detection via DINOv2 nearest neighbor | MVTec-AD | AUROC 96.6% (1-shot) | Yes | N/R |
| CLIP-DINOv2 [19] | 2025 | Zero-shot anomaly detection with DMA fusion of CLIP and DINOv2 | MVTec AD, VisA | AUROC 93.4% / AP 94.3% | No | 23 FPS |
| SAM2-UNet [20] | 2026 | U-shaped segmentation using SAM2 Hiera encoder with PEFT adapters | 18 datasets (5 benchmarks) | mDice 92.8% (Kvasir) | Yes | N/R |
| Semi-sup. defect seg. [21] | 2024 | Two-branch cross pseudo-supervision with DeepLabv3Plus | KolektorSDD, Steel | mIoU +4.39% over SOTA | No | 64.59 FPS |
| Camera Calib. Survey [22] | 2025 | Survey of DL-based camera calibration methods | Holistic calib. dataset | N/R (survey) | Yes | N/R |
| MVTec AD 2 [23] | 2025 | Industrial anomaly detection benchmark with 8 challenging scenarios | MVTec AD 2 | AU-PRO <60% (SOTA) | No | N/R |

Table 2: Summary of the reviewed methods for anomaly detection and segmentation foundations. N/R = Not Reported.

## 2.2 Classical Computer Vision

Yesmin et al. [24] propose a classical image segmentation method for liver cancer detection based on a modified marker-controlled watershed algorithm combined with Sobel edge detection. The approach applies preprocessing steps including grayscale conversion, gradient magnitude computation, and morphological operations to extract foreground and background markers, reducing over-segmentation and noise. Evaluated on MRI images from the LiTS17 dataset, the method shows improvements in PSNR, SNR, and MSE, although its reliance on handcrafted preprocessing limits robustness across diverse imaging conditions.

Liang et al. [6] propose a noise-robust edge detection method based on multi-scale anisotropic morphological Gaussian kernels, designed to overcome limitations of classical detectors such as Canny in noisy environments. The approach introduces an AMDD framework to capture multi-directional intensity variations and combines edge strength and direction maps to improve localization and robustness. Evaluated on BSDS500, NYUDv2, and PASCAL VOC, the method achieves competitive precision–recall scores, outperforming classical detectors while remaining comparable to state-of-the-art techniques.

Sen et al. [25] propose a classical image classification framework based on the fusion of

handcrafted features, combining permutation entropy (PE), Histogram of Oriented Gradients (HOG), and Local Binary Patterns (LBP) with an SVM classifier. The method achieves competitive performance on Fashion-MNIST, KMNIST, EMNIST, and CIFAR-10, offering improved interpretability and computational efficiency compared to deep learning approaches.

Xu [26] presents an overview of automated image processing for industrial inspection, combining classical techniques such as noise filtering, histogram equalization, adaptive thresholding, and edge detection with machine learning-based classification using GLCM and LBP descriptors. While the approach is adaptable to complex industrial conditions, it relies heavily on system design and parameter tuning, which may limit generalization.

Song and Wen [27] propose a 3D modeling method that combines Bezier curves with an improved Harris corner detector. The pipeline includes denoising, feature extraction using an enhanced CSS-Harris algorithm, contour generation via Bezier curves, and surface reconstruction with Delaunay triangulation, showing strong performance in accuracy and speed for geometric modeling tasks.

Hussain [28] proposes a region-based image segmentation method that combines fuzzy logic with geometric features using an energy-based level-set framework to balance region homogeneity, edge preservation, and boundary smoothness. The method achieves $96.8\%$ accuracy and F1 of $95.1\%$, although it requires parameter tuning and may face scalability challenges in real-time applications.

Almohimeed et al. [29] propose a retinal vessel segmentation method combining region growing with the sandpiper optimization algorithm (SPO) for automatic seed point selection and thresholding. The approach achieves up to $98.7\%$ accuracy on DRIVE, STARE, and CHASE_DB1, although it increases computational complexity and requires parameter tuning.

Jia et al. [30] propose an unsupervised ridge-patterned instance segmentation method for high-resolution 3D point clouds, following a four-stage pipeline of ridge extraction, enhancement, adaptive region growing, and plane fitting. The method achieves up to $95.5$ mIoU and is resolution-independent without requiring training data or GPU, although it may struggle with smooth surfaces and complex thin structures.

Guo et al. [31] propose a machine vision method for sub-pixel accurate measurement of flange disk dimensions, combining an improved Canny edge detector with a refined Zernike moment algorithm for sub-pixel refinement followed by least squares fitting. The method achieves reduced measurement errors compared to traditional approaches, although it increases computational complexity and requires careful parameter tuning.

Zangana et al. [32] present a comprehensive review of edge detection techniques covering classical methods (Sobel, Canny, Prewitt) and advanced approaches based on deep learning, fuzzy logic, and optimization. The study identifies key challenges including noise sensitivity, computational cost, and limited generalization, pointing towards future research in real-time and explainable edge detection systems.

A summary of the reviewed methods is provided in Table 3.

| Method | Year | Description | Dataset | Main Metric | Code | Speed |
|---|---|---|---|---|---|---|
| Modified Watershed [24] | 2025 | Marker-controlled watershed for segmentation and noise removal | LiTS17 | PSNR, SNR, MSE (improved) | Yes | N/R |
| MAMDD [6] | 2025 | Multi-scale anisotropic morphological Gaussian kernel edge detection | BSDS500, NYUDv2, PASCAL VOC | PR curves, FOM | No | N/R |
| HOG + LBP + Entropy + SVM [25] | 2025 | Feature fusion for classical image classification | CIFAR-10, Fashion-MNIST | 94.81% accuracy | Yes | N/R |
| Industrial Pipeline [26] | 2025 | Automated vision pipeline for industrial inspection | Industrial datasets | N/R | No | N/R |
| Harris + Bezier + Delaunay [27] | 2025 | Corner detection and geometric modeling for 3D reconstruction | 3D object modeling | 120 pts / 3.2 s | Yes | 3.2 s |
| Fuzzy Segmentation [28] | 2025 | Region segmentation using fuzzy logic and geometry | Segmentation datasets | 96.8% accuracy, F1: 95.1% | Yes | 18.5 s |
| SPORG-RBVS [29] | 2024 | Region growing with optimization for retinal vessel segmentation | DRIVE, STARE, CHASE_DB1 | 98.68% accuracy | No | N/R |
| Ridge Segmentation [30] | 2025 | Ridge-pattern segmentation for 3D point clouds | ABC-10, OmniObject3D-8 | 95.52 mIoU | No | N/R |
| Subpixel Edge Detection [31] | 2024 | Canny + Zernike subpixel edge detection for industrial measurement | Industrial measurement | Reduced absolute/relative error | No | N/R |
| Edge Detection Review [32] | 2024 | Review of classical and modern edge detection techniques | Literature | Precision, Recall, F1 | No | N/R |

Table 3: Summary of the reviewed classical computer vision methods. N/R = Not Reported.

Mira et al. [33] present a comparative study between classical and deep learning approaches for olive fly detection, evaluating combinations such as HOG+SVM, LeNet-5 CNN, and PCA+Random Forest. Their experiments show that PCA combined with Random Forest achieves the best performance, reaching up to 99% accuracy and F1-scores around 0.96, while being more efficient for deployment on edge devices compared to CNN-based solutions.

González et al. [34] propose a robot-assisted computer vision system for adaptive edge finishing in industrial components. The method integrates camera calibration, adaptive binarization, and contour matching using Hu moments to compute deviations with respect to CAD models, reducing the contour RMSE from 1.050 mm to 0.383 mm and demonstrating high precision in industrial environments.

Muzakki et al. [35] introduce a classical approach for bone fracture detection in X-ray images using K-Nearest Neighbors combined with HOG, LBP, and SIFT features. Results on 10,580 images indicate that HOG+KNN achieves the best performance at $99.03\%$ accuracy.

Liu et al. [36] propose a lightweight defect detection framework based on HOG feature extraction, PCA dimensionality reduction, and a LightGBM classifier within a grid-based detection approach. The method achieves up to $98\%$ accuracy with low memory consumption and 0.05 s inference time, making it suitable for resource-constrained environments.

Eryadi [37] conducts a comparative analysis of crack detection methods using HOG features combined with SVM and XGBoost classifiers. Experiments on 40,000 images show that XGBoost achieves the highest accuracy ($99.05\%$) and faster inference times, while SVM produces fewer false positives.

Hung et al. [38] propose a tool wear classification system combining GLCM texture features with SVM and CNN models. CNN models achieve over $93\%$ accuracy, while SVM performance drops to around $70\%$ in complex scenarios, highlighting the limitations of classical methods for highly variable patterns.

Priananda et al. [39] develop a classical pipeline for phase-resolved partial discharge (PRPD) pattern classification using HOG features, PCA, and ensemble classifiers. Bagging Trees achieves $89.79\%$ accuracy, demonstrating the effectiveness of classical pipelines in industrial diagnostic tasks.

Liu et al. [7] propose an improved Canny-based edge detection method incorporating adaptive filtering, Scharr gradient operators, and Otsu thresholding. The approach achieves an average precision of up to 0.92 with improved robustness to noise compared to traditional edge detectors.

Ding et al. [40] present a closed-form solution for homography decomposition using geometric constraints derived from three 3D correspondences. The method achieves accuracy comparable to traditional SVD-based approaches while being 4–7 times faster and more robust in challenging scenarios.

Zhang et al. [41] propose a camera calibration method based on a single checkerboard image using a decoupled parameter estimation strategy, improving calibration accuracy and stability while reducing the complexity of traditional multi-image calibration techniques.

Ge et al. [42] propose a defect classification framework combining improved Canny edge detection, Hu moment features, and an SVM classifier. The method achieves $97.2\%$ accuracy while maintaining low computational cost, offering performance comparable to deep learning approaches.

A summary of the reviewed methods is provided in Table 4.

| Method | Year | Description | Dataset | Main Metric | Code | Speed |
|---|---|---|---|---|---|---|
| HOG + SVM / PCA + RF [33] | 2024 | Classical CV comparison for object detection on edge devices | Olive fly dataset | Accuracy: 99%, F1: 0.96 | No | 9 ms/ROI |
| Adaptive Edge Finishing [34] | 2024 | Robot vision with adaptive binarization and Hu moment contour matching | Aeronautical parts | RMSE: 0.383 mm | No | 31.7 s/image |
| HOG / LBP / SIFT + KNN [35] | 2025 | Classical feature extraction for fracture detection | 10,580 X-ray images | Accuracy: 99.03% | No | N/R |
| Grid-HOG-PCA-LightGBM [36] | 2024 | Lightweight grid-based defect detection | Industrial surface (399 images) | Accuracy: 98% | Yes | 0.05 s |
| HOG + SVM / XGBoost [37] | 2025 | Crack detection with HOG and ensemble classifiers | 40,000 crack images | Accuracy: 99.05% | No | N/R |
| GLCM + SVM / CNN [38] | 2024 | Tool wear classification with texture features | Tool wear dataset | Accuracy: 93%+ (CNN) | No | N/R |
| HOG + PCA + Ensemble [39] | 2024 | PRPD pattern classification with HOG and ensemble classifiers | PRPD dataset | Accuracy: 89.79% | No | N/R |
| Improved Canny [7] | 2024 | Adaptive Canny with Scharr gradients and Otsu thresholding | Rebar dataset | AP: 0.92 | Yes | N/R |
| Homography Decomp. [40] | 2025 | Closed-form homography from geometric constraints | Benchmark dataset | 4–7× faster than SVD | Yes | 4–7× |
| Camera Calibration [41] | 2024 | Single-image calibration with decoupled estimation | Checkerboard dataset | Improved accuracy | Yes | N/R |
| Canny + Hu + SVM [42] | 2025 | Defect classification with edge detection and shape descriptors | Automotive flange | Accuracy: 97.2% | No | N/R |

Table 4: Summary of the reviewed classical computer vision methods with industrial and calibration focus. N/R = Not Reported.

## Comparative Analysis and Method Selection

The reviewed methods span a wide spectrum of approaches, from zero-shot vision-language models to fully classical pipelines. To select the most suitable methods for the Michelin Challenge 2, four primary criteria were applied: (i) **data requirements** — whether the method requires large annotated datasets or can operate in zero-shot or few-shot settings; (ii) **computational cost** — whether the method requires GPU infrastructure or can run on standard CPU hardware; (iii) **applicability to industrial edge detection** — whether the method has been validated in industrial or low-contrast scenarios; and (iv) **integration with geometric calibration** — whether the method can be combined with QR-based metric estimation.

Table 5 summarizes the comparison of the main candidate methods against these criteria.

| Method | Training data | Hardware | Industrial validation | Calibration-compatible |
|---|---|---|---|---|
| Grounded-SAM | Zero-shot | GPU | Partial | Yes |
| YOLOv9 + EfficientSAM | Fine-tuning | GPU | Yes | Yes |
| AnomalyDINO | Few-shot | GPU | Yes (MVTec) | No |
| CLIP-DINOv2 | Zero-shot | GPU | Yes (7 benchmarks) | No |
| MAMDD | None | CPU | Partial | Yes |
| Adaptive Canny pipeline | None | CPU | Yes | Yes |

Table 5: Comparison of candidate methods against the main selection criteria for the Michelin Challenge 2.

Anomaly detection methods such as AnomalyDINO and CLIP-DINOv2, while achieving strong results on industrial benchmarks, are designed to detect deviations from normal patterns rather than to localize and measure geometric structures. This makes them less suitable as primary methods for precise cut edge localization and metric estimation, although they could complement the pipeline for quality control purposes.

Deep learning methods such as Grounded-SAM and YOLOv9 + EfficientSAM offer strong detection and segmentation capabilities but require GPU hardware and, in the case of YOLOv9, labeled training data for fine-tuning. Their main advantage over classical approaches is robustness to appearance variability and the ability to generalize across different acquisition conditions. However, their deployment in a factory environment with limited infrastructure introduces additional complexity.

Classical pipelines based on MAMDD and adaptive Canny edge detection require no training data or GPU resources and offer full interpretability and ease of deployment. Their main limitation is sensitivity to parameter tuning and reduced robustness under large variations in illumination or surface texture compared to end-to-end trained models.

Based on this analysis, four complementary pipelines are proposed in Section 3, covering the full spectrum from zero-shot deep learning to classical CPU-deployable methods, offering flexible trade-offs between accuracy, data requirements, and deployment constraints.

# 3 Selected Method

Grounded-SAM [3] has been selected as the primary deep learning due to its ability to combine open-vocabulary object detection with high-quality segmentation in a unified pipeline.

## 3.1 Grounded-SAM (Deep Learning Approach)

### 3.1.1 Description and motivation

Grounded-SAM [3] is a computer vision framework that combines Grounding DINO, an open-vocabulary object detector, with the Segment Anything Model (SAM) to perform automatic object detection and segmentation. The method enables detecting objects using natural language prompts and generating precise pixel-level segmentation masks, without requiring task-specific retraining.

This method was selected for the Michelin Challenge 2 because the combination of detection and segmentation in a single pipeline allows the system to:

- detect QR codes for spatial calibration,
- segment the table or conveyor belt area,
- detect and segment rubber strip cut edges,
- estimate distances between detected elements using geometric analysis.

### 3.1.2 Architecture

Grounded-SAM has a modular design composed of two main deep learning models working sequentially.

**Grounding DINO (Object Detection Stage).** Grounding DINO is a transformer-based object detector that performs open-vocabulary detection using natural language prompts. The transformer aligns visual features with text embeddings through cross-attention mechanisms, allowing the model to associate image regions with semantic concepts. Its main components are:

- **Backbone network:** Extracts multi-scale visual features from the input image.
- **Text encoder:** Encodes the input text prompt (e.g., "QR code" or "rubber strip edge") into text embeddings.
- **Transformer encoder-decoder:** Processes visual features and aligns them with text embeddings through cross-attention layers.
- **Detection head:** Predicts bounding boxes and confidence scores corresponding to the detected objects.

The output of this stage is a set of bounding boxes associated with the objects described in the text prompt.

**Segment Anything Model (SAM) (Segmentation Stage).** SAM performs high-quality segmentation based on input prompts. Its architecture includes:

- **Image encoder (Vision Transformer):** Extracts high-level visual feature embeddings from the entire input image. This stage runs only once per image.
- **Prompt encoder:** Encodes prompts such as bounding boxes or points into dense and sparse embeddings.
- **Mask decoder:** Generates the final pixel-level segmentation mask by combining image embeddings with prompt embeddings through cross-attention and upsampling operations.

SAM receives the bounding boxes produced by Grounding DINO as spatial prompts and produces pixel-level segmentation masks for the detected objects.

### 3.1.3 Computer vision pipeline

**Step 1: Image acquisition.** An RGB image of the inspection area is captured using a smartphone. Since images are taken by different workers at variable positions and angles, the system must be robust to perspective distortion and changes in illumination.

**Step 2: Preprocessing.** Before feeding the image into the neural networks, the following operations are applied:

- **Image resizing:** The image is resized to match the input resolution expected by each model.
- **Pixel normalization:** Pixel values are normalized to follow the distribution used during training.
- **Tensor conversion:** The image is converted into a PyTorch tensor for neural network processing.
- **QR code detection:** QR codes in the scene are detected and decoded to extract the calibration reference points for metric distance estimation.

**Step 3: Feature extraction.** The preprocessed image is processed by the Grounding DINO backbone, which extracts multi-scale visual feature maps capturing both global and local visual information from the scene.

**Step 4: Object detection with Grounding DINO.** Grounding DINO receives the visual features and a text prompt describing the objects of interest. Examples of prompts used in this application include:

- `"QR code"`

- `"rubber strip"`
- `"cutting edge"`

The output of this stage is a set of bounding boxes localizing the objects described by the prompt.

**Step 5: Segmentation with SAM.** The bounding boxes produced by Grounding DINO are passed as spatial prompts to SAM, which generates pixel-level segmentation masks for the detected objects.

**Step 6: Edge extraction and post-processing.** Once segmentation masks are obtained, image processing techniques are applied to extract geometric information:

- contour detection,
- edge extraction,
- boundary analysis.

These techniques allow identifying the exact borders of the segmented rubber strips or cutting edges.

**Step 7: Geometric analysis and metric estimation.** Using the segmented masks and the QR codes as reference markers, the system computes:

- distances between cutting edges,
- distance from the edge to the table border,
- object positions in the coordinate system defined by the QR codes, where point $(0, 0)$ is the centre of the upper-left QR code.

The pixel-to-millimetre conversion is derived from the known physical distance between QR codes (800 mm legs of a right triangle) using homography or affine transformations to correct for perspective distortion.

#### 3.1.4 Output

The output of the Grounded-SAM pipeline includes:

- bounding boxes of detected objects,
- pixel-level segmentation masks of the rubber strip cut edges,
- pixel coordinates of detected edge boundaries,
- metric measurements of edge positions and distances in millimetres.

### 3.1.5 Evaluation metrics

The performance of Grounded-SAM is evaluated using the following metrics. For the object detection stage performed by Grounding DINO: Average Precision (AP) and mean Average Precision (mAP), which evaluate the accuracy of predicted bounding boxes with respect to ground truth annotations. For the segmentation stage performed by SAM: Intersection over Union (IoU) and mask IoU, which measure the overlap between the predicted segmentation mask and the ground truth mask. These performance metrics are described in detail in Section 5

### 3.1.6 Applicability to the Michelin Challenge

Grounded-SAM is particularly suited for the Michelin Challenge 2 because it integrates object detection and segmentation in a unified pipeline without requiring task-specific training data. In this context, the method can detect QR codes for spatial calibration, segment the table or conveyor belt area, identify the cutting edges of rubber strips, and compute metric distances between detected elements. The combination of open-vocabulary detection and precise segmentation enables accurate localization of relevant structures and supports the measurement tasks required in the automated inspection system.

## 3.2 YOLOv9 + EfficientSAM (Deep Learning Approach)

### 3.2.1 Description and motivation

YOLOv9 + EfficientSAM is a two-stage computer vision pipeline that combines a real-time object detector with a lightweight segmentation model to perform accurate detection and pixel-level segmentation of rubber strip cut edges. YOLOv9 [**?**] is a state-of-the-art real-time object detector that introduces Programmable Gradient Information (PGI) and the Generalized Efficient Layer Aggregation Network (GELAN) to address the information bottleneck problem in deep networks. EfficientSAM [**?**] is a lightweight variant of the Segment Anything Model (SAM) that leverages masked image pretraining to achieve high-quality segmentation with significantly reduced computational cost.

This method was selected for the Michelin Challenge 2 because the combination of detection and segmentation in a sequential pipeline allows the system to:

- detect QR codes for spatial calibration and perspective correction,
- segment the table and rubber band area,
- detect and segment the cut edges of the rubber strip,
- estimate metric distances between detected elements using geometric analysis.

### 3.2.2 Architecture

The pipeline is composed of two main deep learning models working sequentially.

**YOLOv9 (Object Detection Stage).** YOLOv9 [**?**] is a real-time object detector built on two core contributions:

- **GELAN (Generalized Efficient Layer Aggregation Network):** A lightweight network architecture based on gradient path planning that combines CSPNet and ELAN blocks. GELAN achieves better parameter utilization than depth-wise convolution-based designs while maintaining high inference speed and accuracy.
- **PGI (Programmable Gradient Information):** An auxiliary supervision framework composed of an auxiliary reversible branch and multi-level auxiliary information. PGI ensures that reliable gradient information is available throughout training, preventing information loss caused by the information bottleneck in deep networks. The auxiliary reversible branch is removed at inference time, so it introduces no additional computational cost.

The main components of YOLOv9 are:

- **Backbone (GELAN):** Extracts multi-scale visual features from the input image while retaining complete information through gradient path planning.
- **Neck (PAN):** Aggregates features from different scales to improve detection of objects of varying sizes.
- **Detection head:** Predicts bounding boxes and confidence scores for the detected objects.

The output of this stage is a set of bounding boxes localizing the cut edges in the image.

**EfficientSAM (Segmentation Stage).** EfficientSAM [**?**] is a lightweight variant of SAM obtained through a pretraining method called SAMI (SAM-leveraged Masked Image pretraining). Instead of reconstructing raw pixel values as in standard masked autoencoders, SAMI trains a lightweight Vision Transformer encoder to reconstruct the feature embeddings produced by the SAM ViT-H image encoder. This allows the lightweight encoder to acquire the strong visual representations of SAM at a fraction of the computational cost. Its architecture includes:

- **Lightweight image encoder (ViT-Tiny or ViT-Small):** Pretrained with SAMI to extract high-quality visual feature embeddings from the input image.
- **Prompt encoder:** Encodes spatial prompts such as bounding boxes or points into dense and sparse embeddings.
- **Mask decoder:** Generates the final pixel-level segmentation mask by combining image embeddings with prompt embeddings through cross-attention and upsampling operations.

EfficientSAM receives the bounding boxes produced by YOLOv9 as spatial prompts and generates pixel-level segmentation masks for the detected cut edges.

### 3.2.3 Computer vision pipeline

**Step 1: Image acquisition.** An RGB image of the inspection area is captured using a smartphone. Since images are taken by different workers at variable positions and angles, the system must be robust to perspective distortion and illumination changes.

**Step 2: Preprocessing.** Before feeding the image into the neural networks, the following operations are applied:

- **QR code detection and decoding:** The three QR codes present in the scene are detected and decoded using OpenCV. Their pixel coordinates are extracted to define the spatial reference system, where point $(0, 0)$ corresponds to the centre of the upper-left QR code.
- **Homography estimation:** Using the known physical distance between QR codes (800 mm legs of a right triangle) and their detected pixel positions, a homography matrix is computed to correct perspective distortion and derive the pixel-to-millimetre conversion factor.
- **Image resizing and normalization:** The image is resized to match the input resolution expected by each model, pixel values are normalized, and the image is converted into a PyTorch tensor.

**Step 3: Feature extraction.** The preprocessed image is processed by the YOLOv9 GELAN backbone, which extracts multi-scale visual feature maps capturing both global and local information from the scene.

**Step 4: Object detection with YOLOv9.** YOLOv9 processes the extracted features and predicts bounding boxes for the objects of interest. In this application, the model is fine-tuned on the Michelin dataset to detect:

- QR codes (used as calibration markers),
- the rubber strip,
- the cut edges of the rubber strip.

The output of this stage is a set of bounding boxes localizing the cut edges in pixel coordinates.

**Step 5: Segmentation with EfficientSAM.** The bounding boxes produced by YOLOv9 are passed as spatial prompts to EfficientSAM, which generates pixel-level segmentation masks for the detected cut edges. EfficientSAM processes each bounding box independently and outputs a binary mask indicating the exact pixel region corresponding to each cut edge.

**Step 6: Edge extraction and post-processing.** Once segmentation masks are obtained, classical image processing techniques are applied to extract geometric information:

- contour detection,
- edge extraction,
- boundary analysis.

These operations allow identifying the precise boundary coordinates of each segmented cut edge.

**Step 7: Geometric analysis and metric estimation.** Using the segmented masks and the homography derived from the QR codes, the system computes:

- the position of the cut edges in the coordinate system defined by the QR codes, where point $(0, 0)$ is the centre of the upper-left QR code,
- the distance from each cut edge to the lower border of the table,
- the distance between the two cut edges.

All measurements are expressed in millimetres using the pixel-to-millimetre conversion factor estimated in the preprocessing step.

#### 3.2.4 Output

The output of the YOLOv9 + EfficientSAM pipeline includes:

- bounding boxes of the detected cut edges,
- pixel-level segmentation masks of the rubber strip cut edges,
- pixel coordinates of the detected edge boundaries,
- metric measurements of edge positions and distances in millimetres, referenced to the QR code coordinate system.

#### 3.2.5 Evaluation metrics

The performance of the pipeline is evaluated using the following metrics. For the object detection stage performed by YOLOv9: Average Precision (AP) and mean Average Precision (mAP), which evaluate the accuracy of predicted bounding boxes with respect to ground truth annotations. For the segmentation stage performed by EfficientSAM: Intersection over Union (IoU) and mask IoU, which measure the overlap between the predicted segmentation mask and the ground truth mask. These performance metrics are described in detail in Section 5.

### 3.2.6 Applicability to the Michelin Challenge

YOLOv9 + EfficientSAM is well suited for the Michelin Challenge 2 because it provides a complete and efficient pipeline for detection, segmentation and metric estimation. YOLOv9 delivers accurate and fast detection of cut edges even under variable illumination and perspective conditions, while EfficientSAM produces precise pixel-level segmentation masks at a fraction of the computational cost of the original SAM. The use of QR codes as calibration markers and the homography-based perspective correction ensure accurate metric estimation regardless of the camera position. Together, these components enable reliable localization and measurement of rubber strip cut edges in the automated inspection system.

## 3.3 Noise-Robust Edge Detection (Classical Approach)

### 3.3.1 Description and motivation

The selected method is a classical edge detection framework designed to improve robustness against noise and irregular textures by combining multi-scale processing with anisotropic morphological operations. Unlike traditional gradient-based detectors such as Canny or Sobel, this approach introduces a multi-scale anisotropic morphological directional derivative (AMDD) model that captures local intensity variations across multiple orientations.

This method was selected for the Michelin Challenge 2 because the task involves detecting thin separation lines between rubber strips under challenging conditions such as:

- low contrast between adjacent regions,
- surface texture and noise from rubber material,
- varying illumination and acquisition conditions.

The proposed method improves edge continuity and robustness, making it suitable for accurately identifying separation boundaries in industrial environments.

### 3.3.2 Architecture

The method follows a multi-stage classical computer vision pipeline composed of the following components:

**Multi-scale processing.** The input image is processed at multiple scales to capture both fine and coarse edge structures, improving detection robustness across different spatial resolutions.

**Anisotropic morphological filtering.** Directional morphological Gaussian kernels are applied to enhance edge responses along specific orientations while suppressing noise. This allows better discrimination of relevant edges from background texture.

**Directional derivative computation (AMDD).** The anisotropic morphological directional derivative is computed to estimate intensity variations across multiple directions, producing:

- Edge Strength Map (ESM),
- Edge Direction Map (EDM).

**Fusion strategy.** The ESM and EDM are combined to generate a final edge map that improves localization accuracy and continuity.

**Post-processing.** Additional steps such as contrast equalization and spatial filtering are applied to refine edge quality and suppress residual noise.

### 3.3.3 Computer vision pipeline

**Step 1: Image acquisition.** An RGB image of the inspection surface is captured using a mobile device or industrial camera. Images may present perspective distortion, varying illumination, and noise.

**Step 2: Preprocessing.**

- Grayscale conversion,
- Gaussian smoothing,
- Contrast normalization.

**Step 3: Multi-scale feature extraction.** The image is processed at different scales to capture edge structures of varying thickness.

**Step 4: Edge detection (AMDD).** Directional derivatives are computed using anisotropic morphological Gaussian kernels, producing edge strength and direction maps.

**Step 5: Edge fusion.** The edge maps are combined to obtain a refined and continuous edge representation.

**Step 6: Post-processing.**

- Noise suppression,
- Edge thinning,
- Removal of spurious responses.

**Step 7: Line extraction and measurement.** Detected edges are further processed to extract separation lines between rubber strips and compute distances between them.

### 3.3.4 Output

The output of the proposed pipeline includes:

- binary edge map,

- continuous edge contours,
- detected separation lines,
- pixel coordinates of edge boundaries.

#### 3.3.5 Evaluation metrics

The performance of the method is evaluated using standard edge detection metrics:

- Precision and Recall,
- F1-score,
- Pratt's Figure of Merit (FOM),
- Precision-Recall (PR) curves.

These metrics assess both edge localization accuracy and robustness to noise.

#### 3.3.6 Applicability to the Michelin Challenge

The selected method is particularly suitable for the Michelin Challenge 2 because it provides robust edge detection in noisy industrial environments without requiring training data or GPU resources. Its multi-scale and anisotropic design allows accurate detection of thin separation lines between rubber strips, even under low contrast and irregular surface conditions. This makes it an efficient and deployable solution for automated inspection systems, where reliability, interpretability, and computational efficiency are critical.

### 3.4 Adaptive Classical Vision Pipeline for Cut-Edge Detection and Metric Estimation

#### 3.4.1 Description and Motivation

The selected method is an adaptive classical computer vision pipeline for industrial edge detection and metric estimation, built upon the improved Canny edge detection framework proposed by Liu et al. [7] as its primary methodological foundation. This approach was selected because it directly addresses the core challenge of the Michelin Challenge 2, namely the detection of thin, approximately linear cut edges on a rubber strip, without requiring any training data or GPU-based infrastructure. As a result, it can be deployed immediately in industrial environments with minimal setup.

The pipeline is extended with techniques drawn from additional reviewed works: adaptive image segmentation and contour smoothing from González et al. [34], perspective correction and pixel-to-millimetre calibration from Ding et al. [40] and Zhang et al. [41], and a grid-based region of interest strategy from Liu et al. [36]. This combination enables the method to cover the complete processing chain required by the challenge, from raw smartphone image acquisition to metric estimation of cut-edge positions in millimetres.

### 3.4.2 Computer Vision Pipeline

The proposed pipeline consists of five sequential stages:

**Step 1 — Image preprocessing** The input is a raw RGB image captured with a smartphone under varying lighting conditions and camera perspectives. The image is first converted to grayscale, reducing computational complexity. Following González et al. [34], Contrast Limited Adaptive Histogram Equalization (CLAHE) is applied instead of global histogram equalization. This technique operates on local regions, preventing over-amplification of noise in uniform areas, which is crucial in uncontrolled industrial lighting conditions. Finally, a Gaussian blur with a $5 \times 5$ kernel is applied to suppress high-frequency noise prior to edge detection, following standard Canny-based practices [7].

**Step 2 — QR code detection and perspective rectification** The three QR codes present in the image are detected using the `pyzbar` library, which relies on classical binarization and pattern matching. The pixel coordinates of the QR code centres are used to compute a homography matrix via `cv2.findHomography` with RANSAC-based outlier rejection, following the approach of Ding et al. [40]. Given that the QR codes form a right triangle with known dimensions (800 mm), the homography maps image coordinates to a metric coordinate system where $(0, 0)$ corresponds to the upper-left QR code.

The image is then rectified using `cv2.warpPerspective`, correcting perspective distortion caused by camera positioning. Additionally, lens distortion is corrected using the single-image calibration method proposed by Zhang et al. [41], which estimates radial and tangential distortion parameters.

**Step 3 — Adaptive Canny edge detection** Edge detection is performed using an improved version of the Canny operator based on Liu et al. [7]. Instead of manually selecting thresholds, Otsu's method is used to compute an optimal threshold $t_{\text{Otsu}}$ from the grayscale histogram. The Canny thresholds are then defined as:

$$t_{\text{low}} = 0.5 \cdot t_{\text{Otsu}}, \quad t_{\text{high}} = t_{\text{Otsu}}$$

This adaptive strategy ensures robustness to variations in illumination and contrast. Internally, the Canny algorithm computes gradients using Sobel operators, applies non-maximum suppression to obtain thin edges, and performs hysteresis thresholding to retain strong and connected edge responses.

**Step 4 — Region of interest extraction and edge filtering** The binary edge map produced by the Canny detector contains edges from multiple structures. To focus on relevant regions, a grid-based region of interest (ROI) strategy is applied, inspired by Liu et al. [36], where regions with low edge density are discarded.

Within the selected regions, the Probabilistic Hough Transform (`cv2.HoughLinesP`) is used to extract line segments. A geometric filtering stage is then applied: segments

are filtered based on minimum length, orientation, and spatial position relative to the QR-based coordinate system. Collinear segments within a proximity threshold are merged into a single representative line, following a contour smoothing strategy similar to González et al. [34].

**Step 5 — Metric estimation** Once the cut-edge lines are identified, 10 equally spaced sampling points are extracted along each detected edge. Since the image has been rectified into a metric coordinate system, pixel coordinates correspond directly to physical positions in millimetres. The distance from each sampling point to the lower border of the table is computed using Euclidean distance in this metric space.

### 3.4.3 Output

The pipeline produces the following outputs:

- Pixel coordinates of the detected cut-edge boundaries.
- Metric positions of the edges in the QR-based coordinate system, where $(0, 0)$ corresponds to the upper-left QR code.
- Ten distance measurements per edge, corresponding to sampling points, expressed in millimetres.

### 3.4.4 Evaluation Metrics

The performance of the pipeline is evaluated using several metrics. For edge detection accuracy, the Root Mean Square Error (RMSE) between detected edges and ground truth annotations is used, following González et al. [34]. For segmentation quality, Intersection over Union (IoU) is computed between predicted and ground truth masks. Precision and Recall are also reported to assess the trade-off between false positives and false negatives.

Processing speed is measured in seconds per image on a standard CPU, evaluating the practical feasibility of deploying the pipeline in real industrial environments.

### 3.4.5 Applicability to the Michelin Challenge

The proposed pipeline is particularly suitable for the Michelin Challenge 2 as it relies entirely on classical image processing techniques implemented in OpenCV and standard Python libraries. It enables QR-based spatial calibration, perspective correction, robust edge detection under varying conditions, and accurate metric estimation without requiring training data or specialised hardware.

The adaptive thresholding strategy ensures robustness to variations in lighting and camera positioning, while the homography-based transformation allows precise millimetre-level measurements. This makes the method highly practical for deployment in real factory environments using only a smartphone camera.

# 4 Datasets

Several publicly available datasets are commonly used to evaluate object detection and segmentation models such as Grounding DINO and the Segment Anything Model. In addition, this project makes use of a proprietary dataset provided by Michelin Aranda de Duero as part of the challenge.

The **COCO (Common Objects in Context)** dataset was introduced in 2014 by Microsoft [43] and is available at `https://cocodataset.org`. COCO contains more than 330,000 images, with over 200,000 labeled images and more than 1.5 million object instances annotated across 80 object categories, covering common objects in everyday scenes such as people, vehicles, animals, and household items. Each image provides detailed annotations including bounding boxes, instance segmentation masks, and keypoints, making the dataset suitable for evaluating object detection, instance segmentation, and keypoint detection methods. Due to its diversity and scale, COCO is the primary benchmark for evaluating modern detection models such as Grounding DINO and YOLOv8.Two samples from the dataset with their segmentation are shown in Figure 1

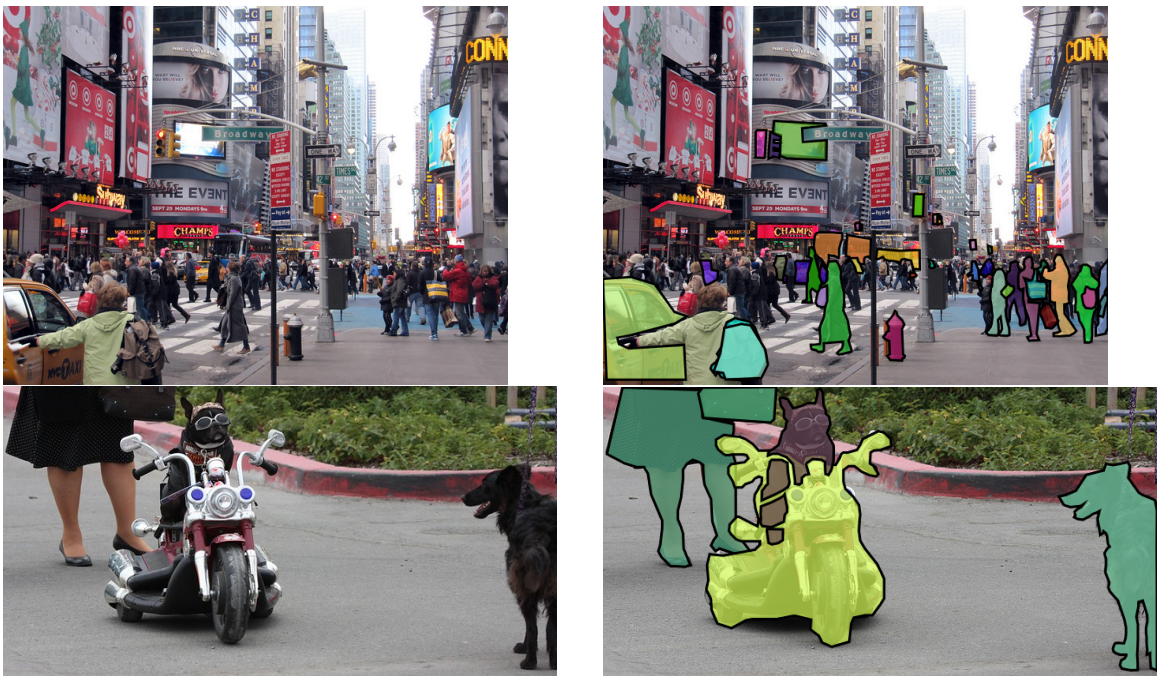


Figure 1: Sample images from the COCO dataset [43]. Left column: original images. Right column: instance segmentation annotations showing object masks across multiple categories.

Another relevant dataset is **NEU Surface Defect Database** was introduced in 2013 by Northeastern University [44] and is available at `https://ieee-dataport.org/documents/neu-det`. The dataset contains 1,800 grayscale images of hot-rolled steel strip surfaces, divided into six categories of surface defects: rolled-in scale, patches, crazing, pitted surface, inclusion, and scratches. Each category contains 300 samples, and the

images have a resolution of $200 \times 200$ pixels. The dataset provides both image-level labels for classification tasks and bounding box annotations for object detection tasks, making it particularly suitable for evaluating defect detection methods in industrial inspection scenarios. Due to its focus on surface defect localization in manufacturing environments, NEU is directly relevant to the Michelin Challenge, where precise detection and localization of structural anomalies in rubber strips is required. Some images of the dataset are shown in Figure 2

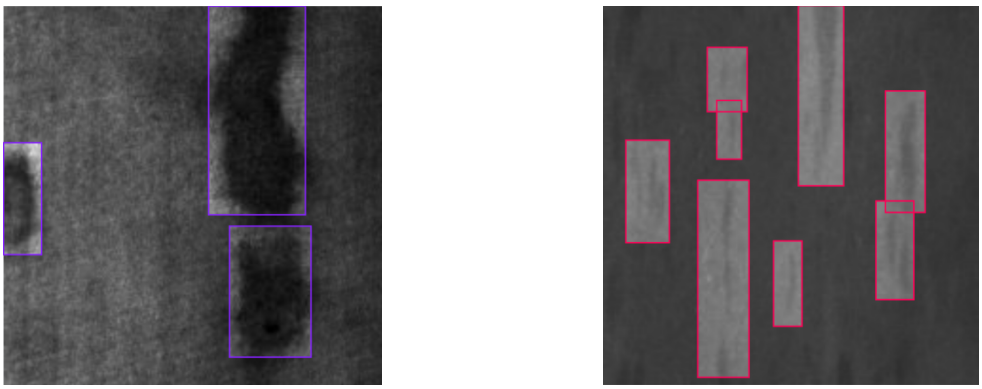


Figure 2: Sample images from the NEU Surface Defect Database [44], showing bounding box annotations for surface defect detection on hot-rolled steel strips.

The **LVIS (Large Vocabulary Instance Segmentation)** dataset was introduced in 2019 by Gupta et al. at Facebook AI Research and is available at its official website[1]. LVIS contains over 164,000 images sourced from the COCO image collection, annotated with high-quality instance segmentation masks across 1,203 object categories, making it one of the most diverse and fine-grained instance segmentation benchmarks available. The dataset is divided into three frequency-based splits: frequent categories (more than 100 training samples), common categories (between 10 and 100 samples), and rare categories (fewer than 10 samples), reflecting the long-tail distribution of object categories in the real world. Annotations were collected using a federated annotation protocol that assigns only a subset of categories to each image, resulting in over 2 million high-quality segmentation masks. LVIS is used to evaluate EfficientSAM [16] on zero-shot instance segmentation, where EfficientSAM-S achieves 42.3 AP on LVIS compared to 44.7 AP for the original SAM, while using $20\times$ fewer parameters. Two sample images from the dataset are shown in Figure **??**.

The **SA-1B** dataset was introduced in 2023 by Kirillov et al. at Meta FAIR and is available at its official website[2]. SA-1B is the largest image segmentation dataset to date, comprising over 11 million high-resolution images with more than 1.1 billion automatically generated segmentation masks. The masks cover objects of all categories without semantic constraints, including whole objects and their parts and subparts, making it a general-

[1] `https://www.lvisdataset.org`
[2] `https://ai.meta.com/datasets/segment-anything/`

purpose dataset for training and evaluating promptable segmentation models. It is used to pretrain and evaluate both SAM 2 [15] and EfficientSAM [16], with the latter achieving 44.4 AP on COCO zero-shot instance segmentation after fine-tuning on SA-1B.

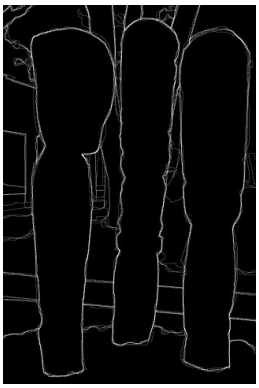
Clase: *pineapple*

Clase: *wine glass*

Figure 3: Sample images from the LVIS dataset with instance segmentation annotations. Each detected instance is represented with a different color mask, allowing individual objects of the same category to be distinguished (*pineapple* and *wine glass*).

The BSDS500 (Berkeley Segmentation Dataset and Benchmark) was introduced by the University of California, Berkeley, and is available at `https://www2.eecs.berkeley.edu/Research/Projects/CS/vision/bsds/` The dataset contains a total of 500 natural images, divided into 200 training, 100 validation, and 200 testing images. Each image is annotated by multiple human subjects, providing ground truth boundaries that reflect perceptual edge information [45, 46].

Original



Detección de bordes

Figure 4: Example of edge detection applied to an original image. The left image shows the original photograph of the sculptures, while the right image presents the result after applying an edge detection algorithm.

Another relevant dataset is PASCAL VOC 2007 (Visual Object Classes), introduced as part of the PASCAL challenge and available at `https://www.robots.ox.ac.uk/`

`~vgg/projects/pascal/VOC/voc2007/` The dataset contains 9,963 images with approximately 24,640 annotated object instances across 20 object categories, including people, vehicles, animals, and everyday objects [47].


Clase: *car* Clase: *aeroplane*

Figure 5: Sample images from the PASCAL VOC dataset with *bounding box* annotations for object detection. Two of the 20 dataset categories are shown: ground vehicles (*car*) and aerial vehicles (*aeroplane*).).

The **IP102** dataset was introduced in 2019 by Wu et al. at Nankai University [48] and is available at `https://github.com/xpwu95/IP102`. The dataset contains more than 75,000 images belonging to 102 insect pest categories, exhibiting a natural long-tailed distribution across species that mainly affect agricultural crops. In addition, approximately 19,000 images are annotated with bounding boxes for object detection tasks. The dataset is organized following a hierarchical taxonomy, grouping pests that affect the same crop into the same upper-level category. IP102 is particularly suitable for evaluating classical and deep feature-based classification methods, including HOG+SVM and PCA+Random Forest pipelines, under realistic inter- and intra-class variance and data imbalance conditions. Samples images from the dataset are shown in Figure 6.

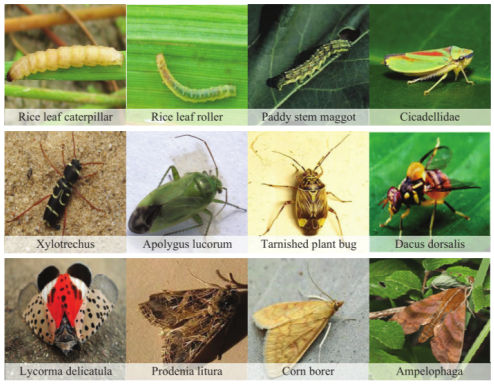



Figure 6: Sample images from the IP102 dataset [48], showing insect pest instances from two different categories. The dataset presents significant inter- and intra-class variance across 102 pest species.

The **MURA (MUsculoskeletal RAdiographs)** dataset, introduced in 2017 by Rajpurkar et al. at Stanford University [49] and available at `https://stanfordmlgroup.github.io/competitions/mura/`. The dataset contains a total of 40,561 musculoskeletal X-ray images from seven anatomical regions: elbow, finger, forearm, hand, humerus, shoulder, and wrist, collected from 14,863 studies. Each image is labelled as either normal or abnormal by board-certified radiologists, making it directly suitable for binary classification tasks such as bone fracture and abnormality detection. Its scale and diversity across anatomical regions make it a standard benchmark for evaluating feature extraction methods such as HOG, LBP, and SIFT combined with classifiers such as K-Nearest Neighbors. Samples images from the dataset are shown in Figure 7.

Figure 7: Sample images from the MURA dataset [49], showing musculoskeletal X-ray studies. Left: normal study. Right: abnormal study with radiologist-confirmed findings.

Lastly, we have **Michelin Aranda Dataset**, which is a propietary image collection provided by Michelin Aranda. The dataset consists of images captured in laboratory conditions simulating real factory scenarios, using a flat table as a surface. Each image contains rubber strips placed on the table alongside three QR codes arranged in a right triangle with legs of 800 mm, used as calibration markers for pixel-to-metric conversion. The images were captured with a smartphone under variable camera positions and angles, reflecting the real acquisition conditions on the production floor. The dataset is specifically designed for the tasks of QR detection, rubber strip segmentation, cut edge localization, and metric distance estimation. Two sample images from this dataset are shown in Figure 8.


Figure 8: Sample images from the Michelin Aranda Dataset. QR codes are visible in the corners of the surface and serve as spatial calibration markers.

## 5 Evaluation Metrics

The performance of the proposed system is evaluated using standard metrics commonly adopted in object detection, image segmentation, and edge detection tasks. The following metrics are used across the four pipelines proposed in this work.

**Precision.** Precision measures the proportion of correctly predicted detections among all predicted detections, reflecting the model's ability to avoid false positives.

$$\text{Precision} = \frac{TP}{TP + FP}$$

**Recall.** Recall measures the ability of the model to detect all relevant objects present in the image, reflecting the model's capacity to minimise false negatives.

$$\text{Recall} = \frac{TP}{TP + FN}$$

**F1-score.** The F1-score provides a balanced measure between Precision and Recall, combining both into a single metric. It is particularly useful when there is an uneven class distribution.

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

**Intersection over Union (IoU).** Intersection over Union evaluates the overlap between the predicted region and the ground truth annotation, defined as the ratio between the intersection area and the union area of both regions.

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

A prediction is typically considered correct if the IoU exceeds a predefined threshold (e.g., 0.5), although stricter evaluations may use multiple thresholds. IoU is used in both object detection and segmentation tasks to assess the quality of predicted masks and bounding boxes.

**Average Precision (AP).** Average Precision summarises the precision-recall curve for a given class by computing the area under the curve, providing a single value representing model performance across different recall levels.

$$\text{AP} = \int_0^1 \text{Precision}(\text{Recall})\, d(\text{Recall})$$

**Mean Average Precision (mAP).** Mean Average Precision is computed as the mean of the Average Precision values across all object classes, and is one of the most widely used metrics in object detection benchmarks.

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^{N} AP_i$$

where $N$ is the number of object classes and $AP_i$ is the Average Precision for class $i$. In this work, mAP is used to evaluate the detection of QR codes and rubber strip cut edges by Grounding DINO and YOLOv9.

**Precision-Recall (PR) Curve.** The PR curve represents the trade-off between Precision and Recall at different classification thresholds. It is widely used in edge detection benchmarks such as BSDS500 to provide a comprehensive view of detector performance across operating points.

**Pratt's Figure of Merit (FOM).** Pratt's FOM evaluates edge localization accuracy by penalising both missed detections and spatial deviations between detected edges and ground truth edges.

$$\text{FOM} = \frac{1}{\max(N_d, N_g)} \sum_{i=1}^{N_d} \frac{1}{1 + \alpha d_i^2}$$

where $N_d$ is the number of detected edge pixels, $N_g$ is the number of ground truth edge pixels, $d_i$ is the distance between a detected edge pixel and the nearest ground truth edge pixel, and $\alpha$ is a scaling constant. FOM is used to evaluate the MAMDD classical edge detector.

**Root Mean Square Error (RMSE).** RMSE measures the average magnitude of errors between predicted metric positions and ground truth measurements, penalising large deviations more heavily than small ones.

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2}$$

where $\hat{y}_i$ is the predicted metric position of the $i$-th point and $y_i$ is the corresponding ground truth value. RMSE is the primary metric for evaluating the millimetre-level accuracy of the adaptive Canny pipeline.

**Mean Absolute Error (MAE).** MAE measures the average absolute difference between predicted and ground truth metric positions. Unlike RMSE, it treats all errors equally regardless of their magnitude, providing a more interpretable measure of average deviation in millimetres.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|\hat{y}_i - y_i|$$

**Distance Error.** The Distance Error evaluates geometric accuracy by measuring the absolute difference between the predicted and ground truth distance between key elements, such as cutting edges and reference QR markers.

$$\text{Distance Error} = |d_{\text{pred}} - d_{\text{gt}}|$$

where $d_{\text{pred}}$ is the estimated distance and $d_{\text{gt}}$ is the ground truth measurement in millimetres. This metric is particularly important for the Michelin Challenge, where precise spatial measurements are required for industrial automation.

In summary, Precision, Recall, F1-score, and PR curves are used to evaluate the quality of edge detection across all pipelines. IoU and mAP assess the accuracy of segmentation masks and object detections respectively. FOM provides a comprehensive assessment of edge continuity and localization accuracy for the classical methods. RMSE, MAE, and Distance Error directly evaluate the precision of millimetre-level geometric measurements, which are the primary output required by the Michelin Challenge.

# 6 Python Implementations

This section presents the available Python implementations of the methods reviewed in Section 2, including their repositories, release dates, main advantages, and limitations, with a focus on their practical applicability to the Michelin Challenge 2.

## 6.1 Deep Learning Methods

**YOLOv8** [1] is available at `https://github.com/ultralytics/ultralytics`. The official implementation was released on January 10, 2023, by Ultralytics as a unified Python

package available via pip, built on PyTorch with both a Python API and a command-line interface. It supports multiple model sizes (n, s, m, l, x) and export to ONNX, TensorRT, and CoreML formats. Its main advantages are ease of use, real-time inference capability, and support for advanced data augmentation strategies such as mosaic and mixup. Its main limitations are a closed vocabulary that restricts detection to predefined categories, and significantly higher computational requirements for larger variants.

**RT-DETRv4** [2] is available at `https://github.com/RT-DETRs/RT-DETRv4`. The repository was created on October 30, 2025, with full code, configurations, and checkpoints released on November 17, 2025. The implementation is provided in PyTorch and includes training, evaluation, and deployment scripts for four model sizes (S, M, L, X) using HGNetv2 as backbone. Its main advantages are high detection accuracy through transformer-based global attention and an end-to-end pipeline that avoids Non-Maximum Suppression. Its main limitations are higher computational requirements than lightweight CNN detectors and greater implementation complexity.

**AnoDDPM** [13] is available at `https://github.com/Julian-Wyatt/AnoDDPM`. The method was introduced at CVPR Workshops 2022 and is written in PyTorch, providing scripts for training on normal data and running anomaly detection at inference time. Note that the repository was archived by the author in October 2025 and is no longer actively maintained. Its main advantages are the ability to learn exclusively from defect-free data and stable training compared to GAN-based approaches. Its main limitations are high computational cost due to the iterative denoising process and the requirement for careful tuning of noise schedules.

**Segformer++** [8] is available at `https://github.com/KieDani/SegformerPlusPlus`. It was introduced in May 2024 and is built on PyTorch with integration with the MMSegmentation and MMPose libraries. A key practical advantage is that the token merging strategy can be applied at inference time using the original Segformer pre-trained weights without any retraining. Its main limitations are that performance gains are less pronounced at lower resolutions, and the method is specifically adapted to the Segformer architecture.

**PEM** [9] is available at `https://github.com/NiccoloCavagnero/PEM`. It was introduced on February 29, 2024, and is provided in PyTorch with support for both semantic and panoptic segmentation under a single unified codebase. Its main advantages are significantly faster inference than comparable models such as Mask2Former and a unified framework that avoids task-specific architectures. Its main limitations are that performance falls slightly below the most accurate but slower models on some benchmarks, and computational requirements remain higher than lightweight CNN-only architectures.

**Relation-DETR** [10] is available at https://github.com/xiuqhou/Relation-DETR. It was introduced on July 16, 2024, and is provided in PyTorch with full training and evaluation scripts for COCO 2017. The position relation encoder is designed as a plug-in-and-play module compatible with any DETR-based architecture. Its main advantages are significantly faster convergence and state-of-the-art detection performance. Its main limitations are increased memory requirements at training time and limited validation on domain-specific datasets.

**ViLU-Net** [11] is available at https://github.com/moeinheidari7829/Retroperitoneal_Tumour_Segmentation_SPIE2025. It was introduced on February 1, 2025, and is built on top of the nnU-Net framework in PyTorch. Its main advantages are state-of-the-art performance on medical segmentation benchmarks and efficient long-range dependency modeling via xLSTM blocks. Its main limitations are the small in-house dataset used for evaluation, high GPU memory requirements (NVIDIA V100, 32 GB), and limited validation beyond CT imaging.

**Grounded-SAM** [3] is available at https://github.com/IDEA-Research/Grounded-Segment-A It was introduced on January 25, 2024, and is actively maintained. The implementation follows a training-free pipeline combining Grounding DINO with SAM, and includes ready-to-use scripts for multiple pipelines including integration with HQ-SAM, MobileSAM, and EfficientSAM. Its main advantages are open-vocabulary detection and segmentation without retraining and strong zero-shot performance on the SegInW benchmark. Its main limitations are inference speed bottlenecks for real-time applications and dependence on the quality of Grounding DINO detections.

**YOLO-World** [12] is available at https://github.com/AILab-CVC/YOLO-World. It was introduced on January 30, 2024, and is built on top of the MMYOLO and MMDetection toolboxes in PyTorch. It provides three model variants (S, M, L) and supports pre-training on large-scale datasets as well as fine-tuning for downstream tasks. Its main advantages are real-time open-vocabulary detection at 52 FPS and a lightweight architecture with as few as 13M parameters. Its main limitations are lower zero-shot performance than heavier open-vocabulary detectors and high pre-training computational requirements.

**Grounding DINO 1.5** [14] is available at https://github.com/IDEA-Research/Grounding-DINO-1.5-API. It was introduced on May 16, 2024, and is accessible via API rather than as a fully open-source codebase, with two model variants: the Pro model using a ViT-L backbone for maximum accuracy, and the Edge model using EfficientViT-L1 for real-time inference. Its main advantages are state-of-the-art zero-shot detection performance and support for diverse input modalities. Its main limitations are the closed API restricting reproducibility and customization, and the high computational requirements of the Pro model.

**SAM 2** [15] is available at `https://github.com/facebookresearch/sam2`. The official implementation was released on August 1, 2024, by Meta FAIR as an open-source PyTorch package under an Apache 2.0 license. It supports both image and video segmentation through a unified promptable interface accepting points, bounding boxes, and masks, with multiple model sizes available (Hiera-T, S, B+, L). Its main advantages are strong zero-shot segmentation performance without task-specific training and a streaming memory architecture enabling video processing. Its main limitations are high GPU memory requirements for larger variants and class-agnostic mask outputs that require additional classification logic for specific downstream tasks.

**EfficientSAM** [16] is available at `https://github.com/yformer/EfficientSAM`. The implementation was released in December 2023 by Meta AI Research in PyTorch, providing two model variants: EfficientSAM-Ti (10M parameters) and EfficientSAM-S (25M parameters), both pretrained via SAMI on SA-1B. Its main advantages are a $20\times$ reduction in parameter count and inference time compared to SAM ViT-H while maintaining competitive segmentation quality. Its main limitations are reduced documentation compared to SAM 2, the absence of video segmentation support, and lower performance on fine-grained segmentation tasks.

**YOLOv9** [17] is available at `https://github.com/WongKinYiu/yolov9`. The implementation was released on February 29, 2024, in PyTorch with full training, evaluation, and inference scripts for MS COCO, providing multiple model sizes (S, M, C, E) and supporting both object detection and instance segmentation. Its main advantages are state-of-the-art accuracy among train-from-scratch detectors and well-documented training pipelines that facilitate fine-tuning on custom datasets. Its main limitations are that the PGI auxiliary branch adds training complexity, and the repository has seen reduced maintenance activity compared to the Ultralytics ecosystem.

**YOLOv8-HDSA** [18] is based on the Ultralytics YOLOv8 framework, available at `https://github.com/ultralytics/ultralytics`. The base framework was released in January 2023 and is actively maintained, with the HDSA-specific modifications introduced in the paper published in March 2025. Its main advantages are an exceptionally mature ecosystem with extensive documentation and easy fine-tuning on custom datasets via a single configuration file. Its main limitation is that the HDSA-specific modifications (SRFF module, convolutional self-attention head, Focaler-IoU loss) are not released separately and must be reimplemented from the paper description.

**AnomalyDINO** [5] is available at `https://github.com/SimonDamm/AnomalyDINO`. The implementation was released in May 2024 and accepted at WACV 2025, leveraging DINOv2 pretrained weights from `torch.hub` without requiring additional training data

beyond a few reference normal images. Its main advantages are a completely training-free inference pipeline and state-of-the-art one-shot performance achieving $96.6\%$ AUROC on MVTec-AD. Its main limitations are performance degradation when fewer than 3–5 reference images are available and sensitivity to domain shift between reference and test images.

**CLIP-DINOv2** [19] does not have a dedicated public repository. The paper, published in December 2025, provides full implementation details including architecture specifications, hyperparameters, and training procedures. The method relies on publicly available pretrained weights for CLIP ViT-L/14@336px from `https://github.com/openai/CLIP` and DINOv2 ViT-L from `https://github.com/facebookresearch/dinov2`. Its main advantages are state-of-the-art zero-shot anomaly detection performance across seven industrial benchmarks. Its main limitation is the absence of a public codebase, requiring full reimplementation from the paper to reproduce results.

**SAM2-UNet** [20] is available at `https://github.com/WZH0120/SAM2-UNet`. The peer-reviewed version was published in Visual Intelligence in January 2026, implemented in PyTorch and requiring SAM 2 pretrained Hiera weights from `https://github.com/facebookresearch/sam2`. Its main advantages are parameter-efficient fine-tuning via lightweight adapters that keep the Hiera encoder frozen, enabling training on a single 24 GB GPU and strong generalization across diverse segmentation tasks. Its main limitation is that the current implementation operates at $352 \times 352$ resolution, which may reduce performance on high-resolution images with fine edge details.

**Semi-supervised defect segmentation** [21] does not have a dedicated public repository. The paper was published in Scientific Reports in September 2024, with code available upon request from the corresponding author via `https://doi.org/10.1038/s41598-024-72579-6`. The implementation is based on PyTorch with a modified DeepLabv3Plus backbone using a simplified MobileNetv2 encoder. Its main advantages are a lightweight architecture achieving 64.59 FPS while outperforming state-of-the-art semi-supervised methods and a dynamic threshold strategy that improves pseudo-label quality over training. Its main limitation is the lack of a public repository, which hinders reproducibility.

**Camera Calibration Survey** [22] maintains a curated repository at `https://github.com/KangLiao929/Awesome-Deep-Camera-Calibration`, last revised in February 2025. The repository is not a single codebase but a structured collection of links to calibration method implementations organized by category, alongside a unified holistic calibration dataset. Its main advantage is providing a comprehensive reference for selecting and comparing calibration methods relevant to the QR-based spatial calibration required in the Michelin Challenge. Its main limitation is that it does not provide ready-to-use calibration code and requires selecting and integrating individual method repositories.

**MVTec AD 2** [23] provides the dataset and evaluation scripts at `https://www.mvtec.com/company/research/datasets/mvtec-ad-2`. Released in March 2025 by MVTec Software GmbH, it does not provide a segmentation or detection model but offers a standardized evaluation framework compatible with existing anomaly detection pipelines. Its main advantage is providing a challenging and realistic benchmark covering scenarios not addressed by existing datasets. Its main limitation is that no baseline model is included, requiring users to integrate their own methods for evaluation.

The deep learning methods reviewed span a wide spectrum of approaches. Foundation segmentation models — SAM 2, EfficientSAM, and SAM2-UNet — share a common architectural lineage based on the Hiera encoder and promptable segmentation paradigm, differing mainly in computational efficiency and downstream adaptability. Object detection methods — YOLOv8, RT-DETRv4, Relation-DETR, YOLO-World, Grounding DINO 1.5, and Grounded-SAM — range from efficient real-time CNN-based detectors to transformer-based open-vocabulary models. Industrial defect methods — YOLOv9, YOLOv8-HDSA, AnomalyDINO, CLIP-DINOv2, and the semi-supervised approach — differ fundamentally in their supervision requirements, from fully supervised fine-tuning to completely training-free inference. From the perspective of the Michelin Challenge, where the available dataset is small and annotations are limited, the most practical deep learning approaches are AnomalyDINO for its training-free nature and the Grounded-SAM and YOLOv9 + EfficientSAM pipelines for their publicly available, well-documented, and PyTorch-compatible implementations.

## 6.2 Classical Methods

**Canny Edge Detector** [50] is available at `https://github.com/opencv/opencv`. The implementation is part of the OpenCV library, originally released in 2000 and continuously maintained, accessible in Python through the `cv2.Canny()` function. Its main advantages are simplicity, efficiency, and real-time performance, making it a standard baseline for edge detection. Its main limitations are sensitivity to noise and fragmented edge responses in complex textures such as rubber surfaces.

**Line Segment Detector (LSD)** [51] is available at `https://github.com/opencv/opencv`. The implementation is included in OpenCV via the `cv2.createLineSegmentDetector()` function. LSD directly detects line segments instead of raw edges, which is particularly useful for identifying separation lines. Its main advantages are high speed, no need for parameter tuning, and direct extraction of geometric structures. Its main limitations are sensitivity to weak edges and reduced performance in very low-contrast scenarios.

**Structured Edge Detection** [52] is available at `https://github.com/opencv/opencv_`

extra, implemented via the cv2.ximgproc.createStructuredEdgeDetection() function using pre-trained structured forest models. The method provides improved edge continuity and robustness compared to classical detectors. Its main advantages are better edge quality and robustness to noise. Its main limitations are the need for pre-trained models and slightly higher computational cost compared to Canny.

**Watershed Segmentation** [53] is available at https://github.com/opencv/opencv, implemented via the cv2.watershed() function. It is widely used for region-based segmentation and object separation. Its main advantages are the ability to separate touching regions and its interpretability. Its main limitations are over-segmentation and sensitivity to marker initialization.

**Frangi Filter** [54] is available at https://github.com/scikit-image/scikit-image, implemented via the skimage.filters.frangi function. The method is designed to enhance line-like structures, making it useful for detecting thin separations. Its main advantages are strong performance on thin structures and robustness to noise. Its main limitations are sensitivity to parameter selection and increased computational cost.

**OpenCV** [**?**] is available at https://github.com/opencv/opencv. OpenCV is an open-source computer vision library first released in 2000 and actively maintained. It provides native Python bindings and includes all the classical image processing operations required by the adaptive Canny pipeline, such as Gaussian blurring, Otsu thresholding, Canny edge detection, the Probabilistic Hough Transform, homography computation via cv2.findHomography, perspective rectification via cv2.warpPerspective, and lens distortion correction via cv2.undistort. Its main advantages are maturity, extensive documentation, cross-platform support, and independence from GPU hardware. Its main limitation is the absence of high-level abstractions for pipeline orchestration, requiring manual integration of each processing stage.

**pyzbar** is available at https://github.com/NaturalHistoryMuseum/pyzbar, released in 2016 and actively maintained. This library wraps the ZBar framework and provides a simple interface for detecting and decoding QR codes from images and video streams. Its main advantage is simplicity and reliability without requiring any training data. Its main limitation is reduced robustness under strong perspective distortion or partial occlusions.

**Grid-HOG-PCA-LightGBM** [36] is implemented using scikit-image for HOG feature extraction (https://github.com/scikit-image/scikit-image), scikit-learn for PCA dimensionality reduction (https://github.com/scikit-learn/scikit-learn), and LightGBM (https://github.com/microsoft/LightGBM) for classification. These libraries are widely used, well-documented, and installable via pip without GPU require-

ments. Its main limitation is that HOG feature extraction is sensitive to parameter choices such as cell size and block normalization, requiring tuning for different datasets.

**Improved Canny** [7] can be directly implemented using OpenCV functions `cv2.Canny` and `cv2.threshold` with the `THRESH_OTSU` flag, requiring no additional repository beyond the standard OpenCV distribution. Its main advantage is simplicity and immediate availability. Its main limitation is that Otsu-based thresholding assumes a bimodal intensity distribution, which may not always hold in complex industrial scenes.

**Homography-based calibration** [40, 41] is implemented using `cv2.findHomography` and `cv2.calibrateCamera`, both included in OpenCV. A reference implementation is available at `https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html`. Its main advantage is that the full calibration and rectification pipeline requires no external dependencies. Its main limitation is that calibration accuracy depends on the quality of detected keypoints and may degrade with low-resolution or poorly captured images.

The classical methods reviewed can be grouped into two categories: edge detection and structure extraction. Edge detection methods — Canny, Structured Edge Detection, and the improved Canny pipeline — focus on identifying intensity discontinuities and boundaries, differing mainly in their robustness to noise and the degree of parameter automation. Structure extraction methods — LSD and Frangi — aim to directly capture geometric features such as line segments or tubular structures, which are more suited to detecting separation patterns in industrial surfaces. Watershed segmentation and the homography-based calibration pipeline serve complementary roles: the former for region separation and the latter for metric estimation. Compared to the deep learning methods described above, classical pipelines require no training data or GPU resources and offer full interpretability, but are generally more sensitive to parameter tuning and less robust to large variations in illumination or surface appearance.

# 7 CREDITS

[3] "CRediT (Contributor Roles Taxonomy) was introduced to recognize individual author contributions, reducing authorship disputes and facilitating collaboration". More informally, it is a paragraph usually introduced at the end of research papers that recognizes the individual work of each author.

- **Pablo** contributed to the Investigation of deep learning methods for object detection and segmentation (YOLOv8, RT-DETRv4, Segformer++, PEM, Relation-DETR, ViLU-Net, Grounded-SAM, YOLO-World, Grounding DINO 1.5, and AnoDDPM)

[3] `https://www.elsevier.com/researcher/author/policies-and-guidelines/credit-author-statement`

and participated in Writing – Original Draft, preparing the initial manuscript for the corresponding sections. They also contributed to Resources, collecting and organizing the datasets described in the study, and to Visualization, preparing figures and summary tables.

- **Jaime** contributed to the Investigation of advanced deep learning methods (SAM 2, EfficientSAM, YOLOv9, YOLOv8-HDSA, AnomalyDINO, CLIP-DINOv2, SAM2-UNet, semi-supervised defect segmentation, camera calibration survey, and MVTec AD 2) and participated in Writing – Original Draft for the corresponding sections. Additionally, they served as the **Project POC**, acting as the main point of contact with the professor for doubts and task uploads. They also contributed to Resources and Visualization alongside the rest of the group.

- **Miguel** contributed to the Investigation of classical computer vision methods (modified watershed, MAMDD, HOG+LBP+Entropy+SVM, industrial inspection pipelines, Harris+Bezier+Delaunay, fuzzy segmentation, SPORG-RBVS, ridge segmentation, subpixel edge detection, and edge detection review) and participated in Writing – Original Draft for the corresponding sections. They also contributed to Resources and Visualization alongside the rest of the group.

- **David** contributed to the Investigation of classical methods with industrial and calibration focus (HOG+SVM/PCA+RF, adaptive edge finishing, HOG/LBP/SIFT+KNN, grid-HOG-PCA-LightGBM, improved Canny, homography decomposition, camera calibration, Canny+Hu+SVM, and additional classical pipelines) and participated in Writing – Original Draft for the corresponding sections. They also contributed to Resources and Visualization alongside the rest of the group.

All four members equally contributed in writing, editing, critically and reviewing the manuscript, and shared supervision responsibilities, providing oversight and leadership throughout the research activities.

## References


[1] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics YOLOv8. `https://github.com/ultralytics/ultralytics`, 2023.

[2] Zhangxuan Liao et al. RT-DETRv4: Painlessly furthering real-time object detection with vision foundation models, 2025.

[3] Tianhe Ren, Shilong Liu, Ailing Zeng, Jing Lin, Kunchang Li, He Cao, Jiayu Chen, Xinyu Huang, Yukang Chen, Feng Yan, Zhaoyang Zeng, Hao Zhang, Feng Li, Jie Yang, Hongyang Li, Qing Jiang, and Lei Zhang. Grounded SAM: Assembling open-world models for diverse visual tasks, 2024.

[4] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. Segment anything, 2023.

[5] Simon Damm, Mike Laszkiewicz, Johannes Lederer, and Asja Fischer. Anomaly-DINO: Boosting patch-based few-shot anomaly detection with DINOv2, 2024. Accepted at WACV 2025 (Oral).

[6] X. Liang et al. Noise-robust image edge detection based on multi-scale automatic anisotropic morphological gaussian kernels. *PLOS ONE*, 2025.

[7] others Liu. Adaptive edge detection of rebar thread head image based on improved Canny operator. *IET Image Processing*, 2024.

[8] Daniel Kienzle, Konstantinos Kantarelis, and Kürt Barthel. Segformer++: Efficient token-merging strategies for high-resolution semantic segmentation, 2024.

[9] Niccolò Cavagnero, Gabriele Rosi, Giuseppe Averta, Fabrizio Lamberti, and Tatiana Tommasi. PEM: Prototype-based efficient MaskFormer for image segmentation, 2024.

[10] Xiuquan Hou, Meiqin Ma, Bin Fan, Ping Luo, and Yanfeng Wan. Relation DETR: Exploring explicit position relation prior for object detection, 2024.

[11] Moein Heidari et al. A study on the performance of U-Net modifications in retroperitoneal tumor segmentation, 2025.

[12] Tianheng Cheng, Lin Song, Yixiao Ge, Wenyu Liu, Xinggang Wang, and Ying Shan. YOLO-World: Real-time open-vocabulary object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16901–16911, 2024.

[13] Julian Wyatt, Adam Leach, Sebastian M. Schmon, and Chris G. Willcocks. AnoDDPM: Anomaly detection with denoising diffusion probabilistic models using simplex noise. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 650–656, 2022.

[14] Tianhe Ren et al. Grounding DINO 1.5: Advance the "edge" of open-set object detection, 2024.

[15] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024.

[16] Yunyang Xiong, Bala Varadarajan, Lemeng Wu, Xiaoyu Xiang, Fanyi Xiao, Chenchen Zhu, Xiaoliang Dai, Dilin Wang, Fei Sun, Forrest Iandola, Raghuraman Krishnamoorthi, and Vikas Chandra. EfficientSAM: Leveraged masked image pretraining for efficient segment anything, 2023.

[17] Chien-Yao Wang, I-Hau Yeh, and Hong-Yuan Mark Liao. YOLOv9: Learning what you want to learn using programmable gradient information, 2024.

[18] Runjie Shi, Zhengbao Li, Zewei Wu, Wenxin Zhang, Yihang Xu, Gan Luo, Pingchuan Ma, and Zheng Zhang. An industrial carbon block instance segmentation algorithm based on improved YOLOv8. *Scientific Reports*, 15:8147, 2025.

[19] Junjie Jiang, Zongxiang He, Anping Wan, Khalil AL-Bukhaiti, Kaiyang Wang, Peiyi Zhu, and Xiaomin Cheng. Zero-shot industrial anomaly detection via CLIP-DINOv2 multimodal fusion and stabilized attention pooling. *Electronics*, 14(24):4785, 2025.

[20] Xinyu Xiong, Zihuang Wu, Shuangyi Tan, Wenxue Li, Feilong Tang, Ying Chen, Siying Li, Jie Ma, and Guanbin Li. SAM2-UNet: Segment anything 2 makes strong encoder for natural and medical image segmentation. *Visual Intelligence*, 4:2, 2026.

[21] Chenbo Shi, Kang Wang, Guodong Zhang, Zelong Li, and Changsheng Zhu. Efficient and accurate semi-supervised semantic segmentation for industrial surface defects. *Scientific Reports*, 14:21874, 2024.

[22] Kang Liao, Lang Nie, Shujuan Huang, Chunyu Lin, Jing Zhang, Yao Zhao, Moncef Gabbouj, and Dacheng Tao. Deep learning for camera calibration and beyond: A survey, 2025. v3, last revised February 2025.

[23] Lars Heckler-Kram, Jan-Hendrik Neudeck, Ulla Scheler, Rebecca König, and Carsten Steger. The MVTec AD 2 dataset: Advanced scenarios for unsupervised anomaly detection, 2025.

[24] S. Yesmin et al. Detection segmentation and noise removal process of cancerous image cell of liver through modified marker-controlled watershed approach. In *Proceedings of the corresponding Springer volume*. Springer, 2025.

[25] A. Sen et al. Feature engineering is not dead: Reviving classical machine learning with entropy, hog, and lbp feature fusion for image classification. *arXiv preprint arXiv:2507.13772*, 2025.

[26] X. Xu. The application of automated image processing technology in industrial inspection. *Journal of the corresponding venue*, 2025.

[27] Hua Song and Ziqiao Wen. Interactive design modeling of 3d styling combining bezier curve and harris corner point detection algorithm. *PLOS ONE*, 20(4):e0319323, 2025.

[28] Ibrar Hussain. A stable region-based image segmentation model integrating fuzzy logic and geometric principles. *Acadlore Transactions on AI and Machine Learning*, 4(2):124–136, 2025.

[29] Ibrahim Almohimeed, Mohamed Yacin Sikkandar, A. Mohanarathinam, Velmurugan Subbiah Parvathy, Mohamad Khairi Ishak, Faten Khalid Karim, and Samih M. Mostafa. Sandpiper optimization algorithm with region growing based robust retinal blood vessel segmentation approach. *IEEE Access*, 12:28612–28620, 2024.

[30] Ran Jia, Junpeng Xue, Feifei He, Hongyang Chen, Kelei Wang, Jiashi Zeng, and Yusong Meng. Robust ridge-patterned instance segmentation for high-resolution point cloud measurement. *IEEE Transactions on Instrumentation and Measurement*, 74:2553514, 2025.

[31] Jun Guo, Yang Yang, and Yuxin Chen. Research on sub-pixel accuracy flange disk dimension measurement based on machine vision. *Signal, Image and Video Processing*, 2024.

[32] Hewa Majeed Zangana, Ayaz Khalid Mohammed, and Firas Mahmood Mustafa. Advancements in edge detection techniques for image enhancement: A comprehensive review. *International Journal of Artificial Intelligence & Robotics*, 6(1):29–39, 2024.

[33] José L. Mira, Jesús Barba, Francisco P. Romero, M. Soledad Escolar, Julián Caba, and Juan C. López. Benchmarking of computer vision methods for energy-efficient high-accuracy olive fly detection on edge devices. *Multimedia Tools and Applications*, 83:81785–81809, 2024.

[34] Mikel González, Adrián Rodríguez, Unai López-Saratxaga, Octavio Pereira, and Luis Norberto López de Lacalle. Adaptive edge finishing process on distorted features through robot-assisted computer vision. *Journal of Manufacturing Systems*, 74:41–54, 2024.

[35] Hasan Muzakki et al. Evaluating K-nearest neighbors for X-ray bone fracture detection using HOG, LBP, and SIFT feature extraction methods. *IEEE Access*, 2025.

[36] Hengxu Liu, Yuqing Xu, Jiwang Huang, Qiaozhi Bao, and Zhenhua Wen. Surface defect detection based on machine learning and data augmentation. In *2024 IEEE 6th International Conference on Civil Aviation Safety and Information Technology (ICCASIT)*, pages 1098–1103, 2024.

[37] Hafizh Eryadi. Comparative analysis of SVM and XGBoost classifiers with HOG features for concrete crack detection. *IT Journal Research and Development*, 2025.

[38] others Hung. Tool wear classification based on support vector machine and deep learning models. *Sensors and Materials*, 2024.

[39] others Prianananda. Machine learning-based phase resolved partial discharge pattern classification using HOG and PCA. In *IEEE Conference*, 2024.

[40] others Ding. Homography decomposition revisited. *International Journal of Computer Vision*, 2025.

[41] others Zhang. A high-quality and convenient camera calibration method using a single image. *Electronics*, 13(22):4361, 2024.

[42] others Ge. Research on surface defect classification model of automotive flange disks based on machine vision. *Signal, Image and Video Processing*, 2025.

[43] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755, 2014.

[44] Kechen Song and Yunhui Yan. A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defects. *Applied Surface Science*, 285:858–864, 2013.

[45] David R. Martin, Charless C. Fowlkes, and Jitendra Malik. A database of human segmented natural images and its application to evaluating segmentation algorithms and measuring ecological statistics. In *Proceedings of the 8th IEEE International Conference on Computer Vision*, pages 416–423, 2001.

[46] Pablo Arbeláez, Michael Maire, Charless Fowlkes, and Jitendra Malik. Contour detection and hierarchical image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(5):898–916, 2011.

[47] Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman. The pascal visual object classes (voc) challenge. *International Journal of Computer Vision*, 88(2):303–338, 2010.

[48] Xiaoping Wu, Chi Zhan, Yu-Kun Lai, Ming-Ming Cheng, and Jufeng Yang. IP102: A large-scale benchmark dataset for insect pest recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8787–8796, 2019.

[49] Pranav Rajpurkar, Jeremy Irvin, Aarti Bagul, Daisy Ding, Tony Duan, Hershel Mehta, Brandon Yang, Kexin Zhu, Matthew Lungren, and Andrew Ng. MURA: Large dataset for abnormality detection in musculoskeletal radiographs. *arXiv preprint arXiv:1712.06957*, 2017.

[50] John Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 8(6):679–698, 1986.

[51] Rafael Grompone von Gioi, Jérémie Jakubowicz, Jean-Michel Morel, and Gregory Randall. Lsd: A line segment detector. *Image Processing On Line*, 2:35–55, 2012.

[52] Piotr Dollár and C. Lawrence Zitnick. Structured forests for fast edge detection. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1841–1848, 2013.

[53] Luc Vincent and Pierre Soille. Watersheds in digital spaces: An efficient algorithm based on immersion simulations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 13(6):583–598, 1991.

[54] Alejandro F. Frangi, Wiro J. Niessen, Koen L. Vincken, and Max A. Viergever. Multiscale vessel enhancement filtering. In *MICCAI*, pages 130–137, 1998.